PROGRAMMING & CS FUNDAMENTALS

Async/Await Visual Guide

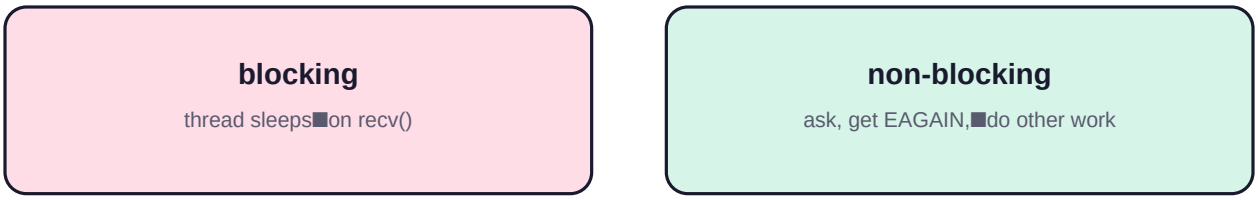
One thread, thousands of tasks —
cooperative concurrency without the thread tax.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Blocking vs Non-Blocking I/O

Waiting is the enemy — async refuses to idle.



1000 blocking connections = 1000 threads = 1000 stacks (~8MB each) = pain.

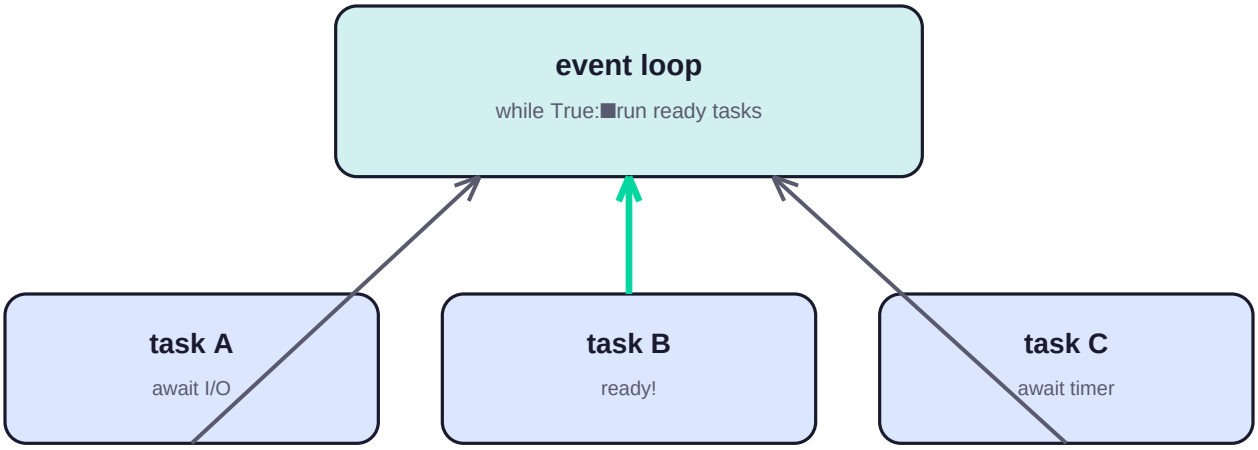
1000 async connections = 1 thread + 1000 tiny task objects = fine.

```
# blocking: the OS parks THIS thread until data arrives
data = sock.recv(1024)          # thread does nothing for 200ms
# async: park the TASK, keep the thread
data = await sock.recv(1024)    # loop runs 999 other tasks meanwhile
```

STEP 2/6

The Event Loop

One scheduler to rule the tasks: run ready ones, park the waiters.



- await = 'I'm stuck; run someone else until my I/O completes'.
- Cooperative: tasks yield VOLUNTARILY — no preemption, no races between awaits.
- The loop also handles timers, signals, and subprocess I/O — one hub for all waiting.

STEP 3/6

Promises / Futures: Values from the Future

A placeholder for a result that hasn't arrived yet.

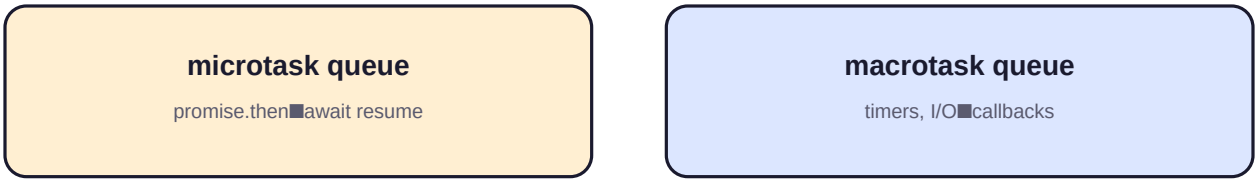
```
p = fetch(url)                  # returns IMMEDIATELY: a promise
p.then(render).catch(show_err) # chain handlers; errors propagate
# async/await is syntax sugar over the same machinery
```

- then() chains SEQUENTIALLY; Promise.all / gather runs CONCURRENTLY.
- Unhandled rejections crash (Node) or warn — always end chains with catch.
- Future vs Promise: naming varies (JS promise, Python future) — the idea is identical.

STEP 4/6

Microtasks vs Macrotasks

Two queues, one rule: drain microtasks first.



Loop tick: run ONE macrotask, then drain ALL microtasks, then render/repeat.

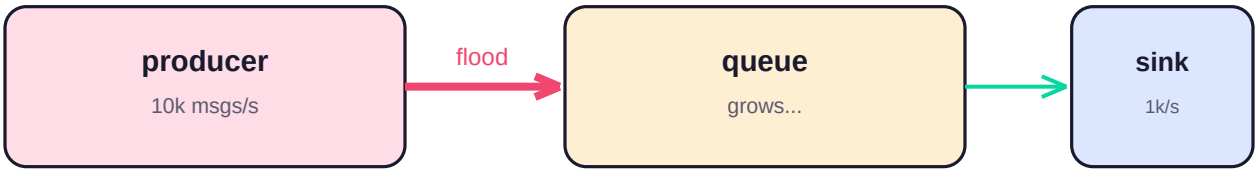
```
setTimeout(() => console.log('macro'), 0)
Promise.resolve().then(() => console.log('micro'))
// prints: micro, THEN macro — always
```

- await continuations are microtasks — they jump ahead of timers.
- Starvation: an infinite microtask chain blocks macrotasks (and rendering) forever.
- queueMicrotask vs setTimeout(0): same 'later', very different priority.

STEP 5/6

Backpressure: When Producers Outrun Consumers

Fast source + slow sink = unbounded memory — unless you push back.




```
# strategies:
# 1. bounded queue + await put()  -> producer WAITS (natural throttle)
# 2. drop newest / oldest         -> lossy, for metrics/logs
# 3. sample / conflate            -> keep latest, for UI state
```

- await queue.put() on a FULL bounded queue IS backpressure — the producer slows itself.
- Unbounded queues are memory leaks with extra steps — always bound or drop.
- Reactive streams (and TCP windows!) are backpressure protocols, not just queues.

STEP 6/6

Async Traps

The ways async code bites.

- Blocking the loop: one time.sleep(5) or sync DB call freezes ALL tasks — use run_in_executor.
- await in a loop when gather() would do: sequential 100 x 100ms = 10s; gathered = ~100ms.
- Fire-and-forget tasks: un-awaited coroutines never run; un-referenced tasks get GC'd mid-flight.
- Shared state between awaits is safe; ACROSS awaits another task may have mutated it — re-check.
- Mixing threads + async: use loop-safe bridges (call_soon_threadsafe), not raw shared queues.
- Async all the way down: one sync function in the chain forces executors or redesign.

STEP ✓I✓

Key Takeaways

Async/Await — Remember

- Async = cooperative concurrency: tasks yield at await; one thread serves thousands.
- Event loop runs ready tasks and parks waiters — never block it with sync calls.
- Promises/futures are future-values with pending/fulfilled/rejected states; chain with then/catch.
- Microtasks (promise continuations) drain before macrotasks (timers, I/O) every tick.
- Backpressure (bounded queues, drop policies) stops fast producers drowning slow consumers.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev